

Rapport d'activité - Rendu 2

Membres du groupe : [Haitam Shaim] et [Abdurrahman Jatri El Marzouqui].

1. Organisation du groupe et méthode de travail : Pour ce second rendu, nous avons maintenu une méthode **fortement collaborative** via GitHub. Nous avons privilégié une approche de conception conjointe pour l'architecture des classes afin de garantir l'interopérabilité entre le moteur de jeu et l'interface graphique. La relecture croisée a été systématique pour valider la gestion des pointeurs intelligents (`std::unique_ptr`) et éviter les fuites de mémoire.

2. Activité individuelle et répartition des initiatives : Afin de garantir une progression cohérente, nous avons réparti les responsabilités tout en maintenant des phases de tests communs :

[Abdurrahman Jatri El Marzouqui] :

- **Initiative et développement principal** : Implémentation de l'interface graphique via **GTKmm**, gestion de la boucle de jeu et de l'interactivité utilisateur (raccourcis 's', 'r', '1' et pilotage de raquette).
- **Développement en binôme** : Intégration du rendu polymorphique dans `Game::draw()` et validation des protocoles de communication entre le moteur et la vue graphique.

[Haitam Shaim] :

- **Initiative et développement principal** : Structuration du modèle de données et des fonctionnalités de persistance (`load`, `save`, `clear`). Mise en œuvre du rendu géométrique dans `tools` et `graphic`.
- **Développement en binôme** : Définition de la hiérarchie des briques et mise en place de l'allocation dynamique sécurisée par pointeurs intelligents pour les objets polymorphiques.

3. Structure du code du modèle :

A. Hiérarchie de classes des briques : Nous avons implémenté une superclasse abstraite **Brick** dont dérivent trois types : `RainbowBrick`, `BallBrick` et `SplitBrick`. La méthode `draw()` est virtuelle, permettant un rendu polymorphique via un seul conteneur.

B. Structuration des entités et classe Game : La classe `Game` centralise les données. La raquette est stockée via `std::unique_ptr<Paddle>`, les balles dans un `std::vector<Ball>`, et les briques polymorphiques dans un `std::vector<std::unique_ptr<Brick>>`.

C. Module tools : Définit les types fondamentaux : `Point`, `Circle`, et `Square`. Il regroupe les fonctions mathématiques et les algorithmes de détection de collision (`intersects`) entre les formes.